Regresión lineal y polinomial

Sesión 4 · Módulo 2: Modelos de regresión

Mínimos cuadrados · Descenso de gradiente · Polinomios y sesgo-varianza · Métricas

Precios de vivienda en California · scikit-learn

Plan de la sesión (≈45 min)

Qué veremos

- **Problema:** predecir el precio de una vivienda (regresión)
- **A.** Mínimos cuadrados ordinarios (OLS): intuición y solución cerrada
- **B.** Descenso de gradiente: por lotes, estocástico y por minilotes
- **C.** Regresión polinomial y el equilibrio sesgo–varianza
- **D.** Métricas: MAE, MSE, RMSE y R^2
- **E.** Caso completo y actividad individual

Regresión: predecir un número

Clasificación vs. regresión

- **Clasificación**: el objetivo es una *categoría* (¿spam o no spam?).
- **Regresión**: el objetivo es un *número* (¿cuánto cuesta esta vivienda?).

La receta de siempre

1. Variables de entrada x (ingreso, edad de las casas, ubicación...)
2. Un modelo f con parámetros que se **aprenden** de los datos
3. Predicción $\hat{y} = f(x)$ y un **error** que medimos

Hoy: el modelo más simple y el más usado

Una **combinación lineal** de las variables. Es rápido, se puede explicar y sirve de **punto de referencia** para cualquier modelo más complejo que veamos después (Ridge, SVR, árboles, ensambles).

Todo lo de hoy tiene una sola idea de fondo: **elegir los parámetros que hacen pequeño el error**. Lo que cambia es cómo los calculamos y qué tan flexible es el modelo.

Nuestro caso: precios de vivienda en California

Variable	Qué describe
`MedInc`	ingreso mediano del distrito ($\times 10\,000$ USD)
`HouseAge`	antigüedad mediana de las casas
`AveRooms`	habitaciones por hogar
`AveOccup`	ocupantes por hogar
`Population`	habitantes del distrito
`Latitude`, `Longitude`	ubicación
`MedHouseVal`	**objetivo**: valor mediano ($\times 100\,000$ USD)

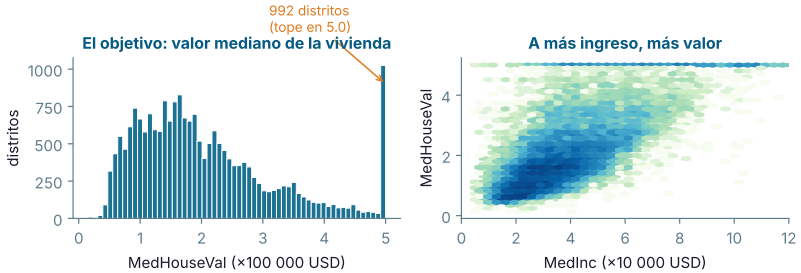
Los datos

- $n = 20\,640$ **distritos** (censo de 1990).
- Quitamos 361 distritos con valores extremos (`AveRooms`≥ 10, `AveOccup`≥ 6 o `Population`$\geq 10\,000$), tal como vimos en el EDA: quedan **20 279**.
- División: 80 % entrenamiento (16 223) y 20 % prueba (4 056).

Ya lo conocemos

Es el mismo conjunto de la Sesión 1; ahora lo usamos para **predecir**.

El objetivo y su relación con el ingreso



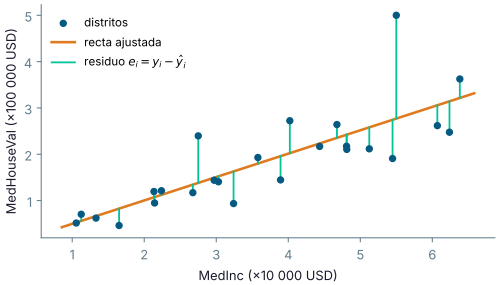
Lo que vemos

El valor medio es 2.07 (unos 207 000 USD). Hay un **tope en 5.0**: 992 distritos están "cortados" en 500 000 USD. Más ingreso se asocia con más valor.

Línea base: el modelo más tonto

Predecir **siempre el promedio** del entrenamiento da en prueba un error medio absoluto de 0.93 y una raíz del error cuadrático de 1.18. **Cualquier modelo útil debe superarlo.**

A1 · Ajustar una recta: ¿cuál es la mejor?



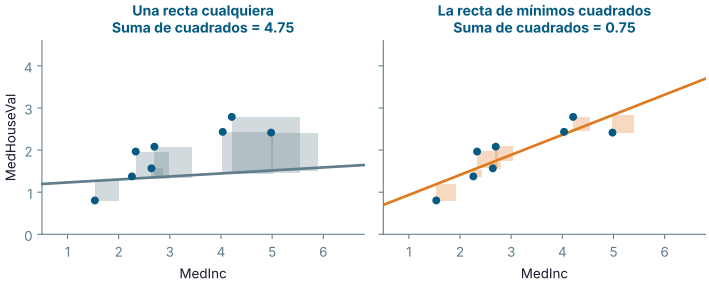
Modelo de una variable

$$\hat{y} = b + w\,x$$

w es la **pendiente** y b el **intercepto**: son los parámetros que el modelo aprende.

- Cada punto queda a cierta distancia vertical de la recta: el **residuo** $e_i = y_i - \hat{y}_i$.
- Una recta es "buena" si los residuos son pequeños *en conjunto*.
- ¿Cómo combinamos 25 residuos en un solo número?

A2 · Mínimos cuadrados: sumar áreas de cuadrados



Criterio de mínimos cuadrados

$$\text{SSE}(b, w) = \sum_{i=1}^{n} \left(y_i - \hat{y}_i\right)^2$$

Cada residuo se eleva al cuadrado (área de un cuadrado). OLS elige b, w con la **menor suma de áreas**.

¿Por qué al cuadrado?

- Los errores positivos y negativos no se cancelan.
- Castiga mucho los errores grandes.
- Da una función suave con **un solo mínimo**, fácil de resolver.

A3 · Solución cerrada: se resuelve de una vez

Con una variable

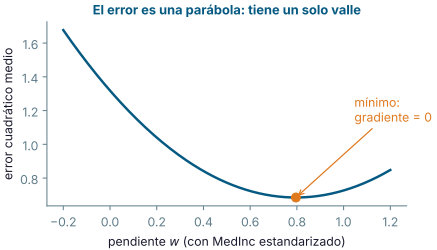
$$w = \frac{\sum_i (x_i - \bar{x})(y_i - \bar{y})}{\sum_i (x_i - \bar{x})^2}, \qquad b = \bar{y} - w\,\bar{x}$$

w es la **covarianza** entre x e y dividida entre la **varianza** de x. La recta siempre pasa por el punto de medias $(\bar{x}, \bar{y})$.

Con varias variables (forma matricial)

$$\hat{\boldsymbol{\beta}} = (X^\top X)^{-1} X^\top \mathbf{y}$$

Ecuación normal: se obtiene igualando a cero el gradiente del error.



El error en función de la pendiente es una **parábola**: en el fondo la pendiente de la curva (el **gradiente**) vale cero. Resolver $\nabla \mathrm{SSE} = 0$ da la fórmula de la izquierda.

A4 · Resultado con una variable: ingreso → valor

Ajuste con `MedInc` (entrenamiento)

$$\widehat{\texttt{MedHouseVal}} = 0.438 + 0.420 \cdot \texttt{MedInc}$$

- **Pendiente**: un aumento de 1 en `MedInc` (10 000 USD de ingreso) se asocia con **+0.42** (42 000 USD) en el valor.
- **Intercepto**: valor predicho con ingreso 0; sirve para colocar la recta, no se interpreta literalmente.

En el conjunto de prueba

	MAE	RMSE	R^2
Siempre el promedio	0.93	1.18	0.00
Recta con `MedInc`	0.64	0.86	0.46

Una sola variable ya mejora mucho

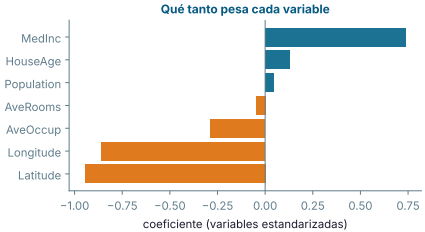
Con una sola columna, el error baja de 1.18 a 0.86. ¿Y si usamos las siete?

A5 · Regresión lineal múltiple: una variable más, un peso más

Modelo

$$\hat{y} = b + w_1 x_1 + w_2 x_2 + \cdots + w_p x_p$$

Cada w_j dice cuánto cambia la predicción al subir x_j una unidad, **manteniendo fijas las demás**.



Lectura de los pesos (variables estandarizadas)

- `MedInc` pesa mucho y en positivo.
- `Latitude` y `Longitude` **negativas**: en promedio, más al norte y más hacia el interior, el valor baja (una tendencia general, no una regla local).
- `AveOccup` negativa: más gente por hogar, menor valor.

Ojo

Para **comparar** pesos hay que estandarizar. Con unidades originales, `Population` tiene un peso casi 0 simplemente porque sus valores son enormes.

A6 · En scikit-learn

```python
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error

Xtr, Xte, ytr, yte = train_test_split(
    X, y, test_size=0.2, random_state=42)

modelo = LinearRegression()       # OLS, solucion cerrada
modelo.fit(Xtr, ytr)

print(modelo.intercept_, modelo.coef_)
y_pred = modelo.predict(Xte)
rmse = mean_squared_error(yte, y_pred) ** 0.5
```

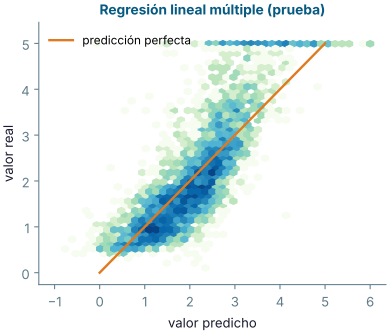
Resultado en prueba

	MAE	RMSE	R^2
Promedio	0.93	1.18	0.00
`MedInc`	0.64	0.86	0.46
7 variables	**0.51**	**0.69**	**0.66**

Se aprende con `fit`

Con 16 223 filas y 7 columnas, el ajuste es inmediato: es una sola operación de álgebra lineal.

A7 · ¿Qué tan bien predice? Y cuidados con OLS



Cuidados al usar OLS

- **Valores atípicos**: un punto lejano jala la recta (el error al cuadrado lo castiga mucho).
- **Variables muy correlacionadas**: los pesos se vuelven inestables, aunque la predicción siga bien.
- **Extrapolar**: fuera del rango de los datos la recta no tiene garantía.
- **Relaciones curvas**: una recta no las capta (lo atacamos en la parte C).

La nube se aleja de la diagonal en los valores altos, y la franja horizontal de arriba son los distritos "cortados" en 5.0.

Pausa para pensar (actividad 1, 2 min)

Trabaja de forma individual

Regresión simple: $\widehat{\texttt{MedHouseVal}} = 0.438 + 0.420 \cdot \texttt{MedInc}$.

1. ¿Qué valor predice para un distrito con $\texttt{MedInc} = 5$? (Escribe tu respuesta en USD.)
2. Si el valor real de ese distrito es 2.9, ¿cuánto vale el residuo?
3. ¿Qué ocurre con la predicción si el ingreso sube 2 unidades?

Compara después con un compañero. Respuestas: predicción $= 0.438 + 2.10 = 2.538$ (253 800 USD); residuo $= 2.9 - 2.538 = 0.362$ (36 200 USD); sube 0.84 (84 000 USD).

B1 · ¿Y si no se puede resolver de una vez?

Límites de la solución cerrada

- Hay que invertir $X^\top X$, de tamaño $p \times p$: se vuelve **costoso** con muchas variables (miles).
- Los datos deben caber **en memoria** de una vez.
- Muchos modelos (redes, regresión logística, boosting) **no tienen** fórmula cerrada.

Idea: bajar la montaña a pasos

Imagina que estás en una montaña con niebla y quieres llegar al valle. No ves el valle, pero **sientes la pendiente bajo tus pies**.

Das un paso en la dirección en que el terreno baja más rápido. Repites hasta que el piso queda plano.

El "terreno" es el error SSE en función de los parámetros; en OLS tiene la forma de un valle (convexo), así que **siempre llegamos al mismo fondo**.

B2 · Descenso de gradiente: la regla de actualización

Regla

$$\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \eta \nabla J(\boldsymbol{\theta})$$

- $\boldsymbol{\theta}$: los parámetros $(b, w_1, \ldots, w_p)$.
- J: el error (aquí el error cuadrático medio).
- ∇J: el **gradiente**, que apunta hacia donde el error *sube*; por eso restamos.
- η: la **tasa de aprendizaje** (tamaño del paso).

Algoritmo

1) Empieza con $\boldsymbol{\theta}$ cualquiera. 2) Calcula el gradiente. 3) Da el paso. 4) Repite hasta que el error deje de bajar.

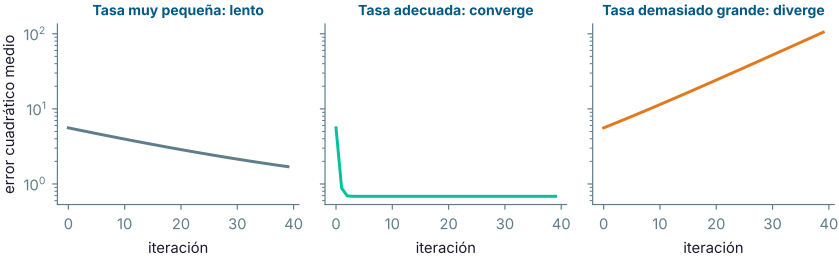
Para regresión lineal

El gradiente tiene forma sencilla:

$$\nabla J = \frac{2}{n} X^\top (X\boldsymbol{\theta} - \mathbf{y})$$

Es el **promedio de los residuos por cada variable**: si una variable está correlacionada con el error, su peso se ajusta más.

B3 · La tasa de aprendizaje η: el tamaño del paso importa



Muy pequeña

Seguro pero **muy lento**: tras 40 iteraciones apenas ha bajado.

Adecuada

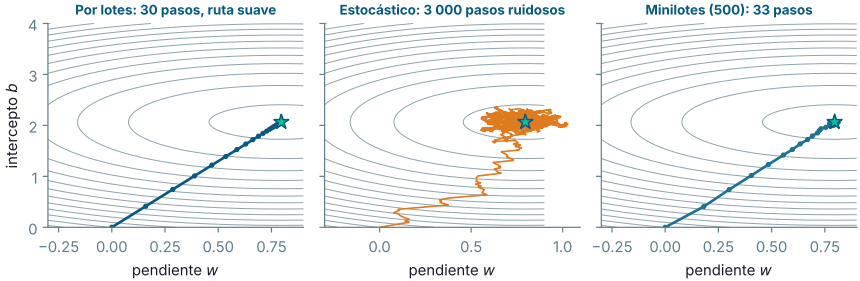
Baja rápido y se **estabiliza** en el mínimo.

Muy grande

Se pasa del valle de un lado al otro y el error **crece**: el algoritmo diverge.

Ejemplo con `MedInc` estandarizado, $\eta = 0.01$, 0.4 y 1.02.

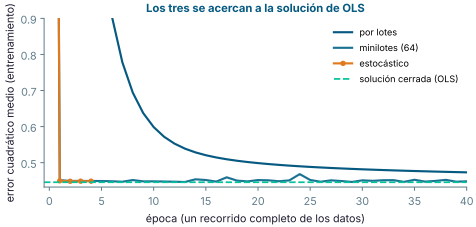
B4 · Tres versiones: ¿con cuántos datos calculamos el gradiente?



Curvas de nivel del error en el plano (pendiente, intercepto). La estrella es la solución de OLS. Mismo punto de partida en los tres casos.

Versión	Datos por paso	Carácter
Por lotes (batch)	los n datos	ruta suave y exacta, pero cada paso es costoso
Estocástico (SGD)	**1** dato	paso barato, ruta ruidosa que "baila" cerca del mínimo
Minilotes	16–512 datos	el compromiso: poco ruido y aprovecha la vectorización

B5 · Cuándo usar cada una



Error de entrenamiento con las 7 variables estandarizadas. Una **época** es un recorrido completo por los datos. El estocástico y los minilotes ya están en 0.45 tras una época; el de lotes llega a 0.473 tras 40 (OLS: 0.446).

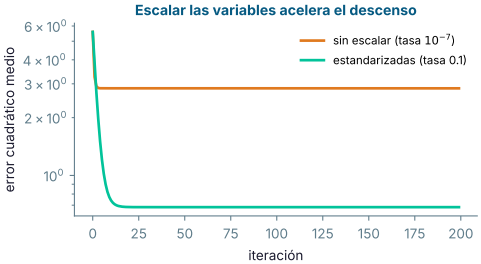
Reglas prácticas

- **Lotes**: datos pequeños; trayectoria estable.
- **SGD**: datos enormes o que llegan en flujo (*online*); el ruido ayuda a salir de mínimos locales en modelos no convexos.
- **Minilotes**: **lo más usado** en la práctica (redes neuronales, boosting con GPU).

Hiperparámetros del descenso

Tasa η, tamaño de minilote y número de épocas. Una tasa que **decrece** con el tiempo reduce el baile del SGD.

B6 · Escalar las variables hace que el descenso funcione



¿Por qué?

`MedInc` va de 0.5 a 15; `Population`, de 3 a casi 10 000. Con escalas tan distintas el terreno es un valle **estrecho y alargado**: el paso que sirve para una variable se pasa para la otra.

Resultado

Tras 200 iteraciones, sin escalar el error es **2.84**; con variables estandarizadas, **0.68** (el mínimo de OLS).

Regla: **siempre** estandariza antes de usar descenso de gradiente (dentro de un `Pipeline`, para evitar fuga de datos).

B7 · En scikit-learn: SGDRegressor

```python
from sklearn.linear_model import SGDRegressor
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

sgd = make_pipeline(
    StandardScaler(),    # indispensable
    SGDRegressor(loss="squared_error",
                 learning_rate="invscaling",  # tasa decreciente
                 eta0=0.01,
                 max_iter=1000, tol=1e-4,
                 random_state=0))
sgd.fit(Xtr, ytr)
print(sgd.score(Xte, yte))      # R^2 en prueba
```

Comparación

- `LinearRegression`: solución cerrada, sin hiperparámetros.
- `SGDRegressor`: mismo modelo, pero iterativo; admite `partial_fit` para datos por partes.

Esperable

Ambos deben dar un R^2 **casi idéntico** (cerca de 0.66): son dos caminos al mismo valle.

C1 · ¿Y si la relación no es una recta?

Regresión polinomial

$$\hat{y} = b + w_1 x + w_2 x^2 + w_3 x^3 + \cdots + w_d x^d$$

Es **creación de variables** (Sesión 3): agregamos $x^2, x^3, \ldots$ como columnas nuevas y ajustamos una regresión lineal con ellas.

Lineal en los parámetros

Aunque la curva sea doblada, el modelo es **lineal en los pesos** w_j: OLS y el descenso de gradiente funcionan igual, sin cambiar nada.

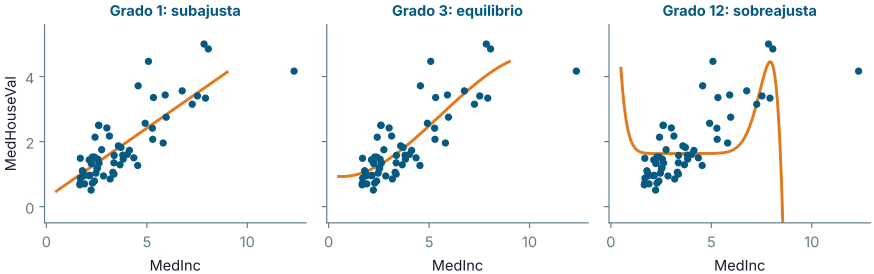
Con varias variables

Con 2 variables y grado 2, aparecen:

$$x_1,\ x_2,\ x_1^2,\ x_2^2,\ x_1 x_2$$

Los términos $x_1 x_2$ son **interacciones**: el efecto de una variable depende del valor de otra (por ejemplo, ingreso y ubicación).

C2 · El grado controla la flexibilidad



60 distritos de entrenamiento (`MedInc` → valor). Polinomio de grado 1, 3 y 12 ajustado con `PolynomialFeatures`.

Grado 1

Demasiado rígido: **subajuste**. Ignora la curvatura.

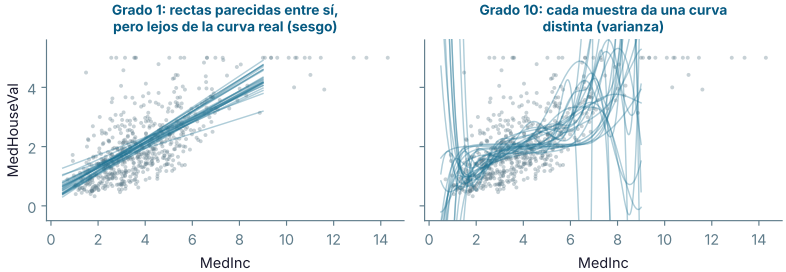
Grado 3

Capta la tendencia sin perseguir cada punto.

Grado 12

Pasa cerca de los datos pero oscila de forma absurda: **sobreajuste**.

C3 · Sesgo y varianza: dos fuentes de error



Cada línea azul es un ajuste con **otra muestra** de 60 distritos. Los puntos grises son los datos disponibles.

Sesgo (bias)

Error por un modelo **demasiado simple**: aunque tuviéramos infinitos datos, no puede captar la forma real. Es el error *sistemático*.

Varianza

Error por un modelo **demasiado sensible** a la muestra: cambia mucho si cambian los datos. Es el error por *azar*.

C4 · El equilibrio: más complejidad baja el sesgo y sube la varianza

Descomposición del error de prueba

$$\text{Error} = \underbrace{\text{Sesgo}^2}_{\downarrow \text{ con el grado}} + \underbrace{\text{Varianza}}_{\uparrow \text{ con el grado}} + \underbrace{\text{Ruido}}_{\text{inevitable}}$$

- **Grado bajo**: sesgo alto, varianza baja. Error de entrenamiento **y** de prueba altos.
- **Grado alto**: sesgo bajo, varianza alta. Error de entrenamiento muy bajo, de prueba **alto**.
- Buscamos el **punto medio**: el mínimo del error de validación.

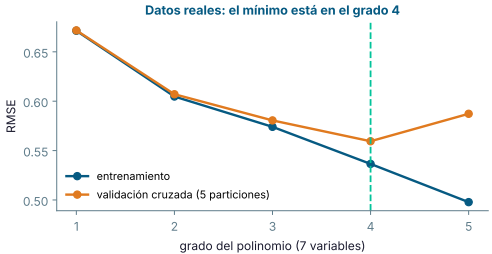
Cómo se reconoce

	Entren.	**Validación**
Subajuste	alto	alto (similar)
Bien	bajo	bajo (cercano)
Sobreajuste	muy bajo	alto (brecha grande)

Lo importante

El error de **entrenamiento** siempre baja con el grado: **no sirve** para elegirlo. Hay que mirar la validación.

C5 · Con los datos reales: ¿qué grado conviene?



Las 7 variables, validación cruzada (20 279 distritos)

Grado	Columnas	RMSE entren.	RMSE valid.
1	7	0.671	0.672
2	35	0.605	0.607
3	119	0.574	0.581
4	329	0.536	**0.559**
5	791	0.498	0.587

Lectura

La validación mejora hasta el **grado 4**; en el 5 el entrenamiento sigue bajando pero la validación **sube**: sobreajuste.

C6 · Cuidados con los polinomios

Las columnas se multiplican rápidamente

Con p variables y grado d hay $\binom{p+d}{d} - 1$ columnas. Con $p = 7$:

Grado	1	2	3	4	5
Columnas	7	35	119	329	791

Receta

`StandardScaler` → `PolynomialFeatures` → `LinearRegression`, todo dentro de un `Pipeline`. Para controlar el sobreajuste con muchas columnas, la próxima sesión veremos **Ridge y Lasso**.

Sin limpiar atípicos, el polinomio explota

Con los **20 640 distritos originales** y grado 3, el error de validación cruzada por partición fue:

347.5, 0.9, 29.1, 41.4, 0.6

Un distrito con `AveOccup` = 1243 se eleva al cubo y su efecto se dispara: **las potencias amplifican los valores extremos**.

Con datos limpios: 0.58 en el grado 3.

C7 · En scikit-learn: un `Pipeline` polinomial

```python
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import (StandardScaler,
                                   PolynomialFeatures)
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import cross_val_score

for grado in range(1, 6):
    modelo = make_pipeline(
        StandardScaler(),
        PolynomialFeatures(grado, include_bias=False),
        LinearRegression())
    rmse = -cross_val_score(
        modelo, Xtr, ytr, cv=5,
        scoring="neg_root_mean_squared_error").mean()
    print(grado, round(rmse, 3))
```

Puntos clave

- `include_bias=False`: `LinearRegression` ya agrega el intercepto.
- Todo va **dentro del pipeline**: el escalado y los polinomios se aprenden solo con los datos de entrenamiento de cada partición.
- Elegimos el grado con **validación cruzada** y el conjunto de prueba se usa **una sola vez** al final.

Pausa para pensar (actividad 2, 3 min)

Trabaja de forma individual

Un compañero ajusta un polinomio de grado 15 y obtiene un error de **entrenamiento de 0.05** y un error de **validación de 0.90**.

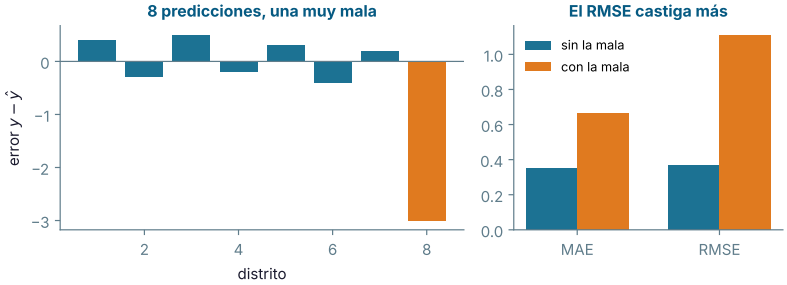
1. ¿Es un caso de subajuste o de sobreajuste? ¿Por qué?
2. ¿Tiene el modelo sesgo alto o varianza alta?
3. Menciona **dos acciones** que podrían ayudar.

Pista: compara los dos errores. Ejemplos de acciones: bajar el grado, conseguir más datos, regularizar (próxima sesión), usar menos variables.

D1 · Cuatro métricas para regresión

Métrica	Fórmula	Cómo se lee
MAE error absoluto medio	$\frac{1}{n}\sum_i \lvert y_i - \hat{y}_i \rvert$	"Me equivoco en promedio por ..." (mismas unidades que y)
MSE error cuadrático medio	$\frac{1}{n}\sum_i (y_i - \hat{y}_i)^2$	lo que minimiza OLS; unidades al cuadrado, difícil de interpretar
RMSE raíz del MSE	$\sqrt{\text{MSE}}$	como el MAE, en las unidades de y, pero **castiga más los errores grandes**
R^2 coeficiente de determinación	$1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2}$	"fracción de la variación de y que el modelo explica"

D2 · MAE vs. RMSE: ¿cuánto castigamos un error grande?



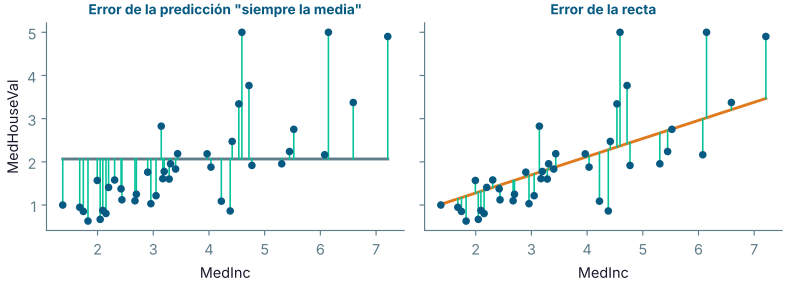
Ocho predicciones

Con errores pequeños (entre 0.2 y 0.5): MAE = 0.35, RMSE = 0.37. Si **una sola** falla por 3: el MAE sube a 0.66 (casi el doble) pero el RMSE sube a **1.11** (el triple).

¿Cuál usar?

MAE si los errores grandes no son especialmente graves y hay atípicos. **RMSE** si un error grande *cuesta mucho más* que varios pequeños. Conviene reportar **ambas**.

D3 · R^2: compararse contra “siempre el promedio”



Interpretación

$R^2 = 1 - \dfrac{\text{error de la recta}}{\text{error de predecir el promedio}}$.

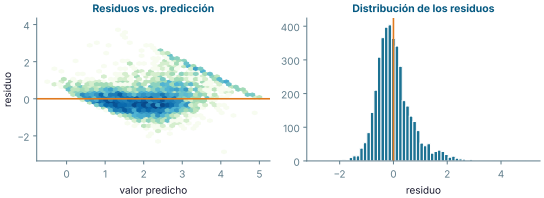
$R^2 = 1$: predicción perfecta. $R^2 = 0$: tan mal como el promedio.

$R^2 < 0$: **peor** que el promedio.

Cuidado

En **entrenamiento**, R^2 nunca baja al agregar variables o grados: aunque el modelo sobreajuste. **Calcúlalo siempre en validación o prueba.**

D4 · Los residuos cuentan lo que las métricas no dicen



Qué buscar

- Residuos **sin patrón**, centrados en 0 (aquí el promedio es 0.02).
- Una **curva** o un abanico indican que falta no linealidad o que la varianza cambia.
- Una cola larga muestra que algunos distritos se predicen muy mal.

Aquí

La banda diagonal corresponde a los distritos con tope en 5.0: el valor real está “cortado” y el modelo no lo sabe.

E1 · Comparación final en el conjunto de prueba

Modelo	Columnas	MAE	RMSE	R^2
Siempre el promedio	0	0.93	1.18	0.00
Recta con `MedInc`	1	0.64	0.86	0.46
Lineal con 7 variables	7	0.51	0.69	0.66
Polinomial grado 2	35	0.44	0.62	0.72
Polinomial grado 3	119	0.42	0.59	0.75
Polinomial grado 4	329	**0.40**	**0.59**	**0.75**

Conclusión

De la recta simple a un polinomio de grado 3–4, el RMSE baja de 0.86 a 0.59 (alrededor de 59 000 USD de error típico). Del 3 al 4 la mejora ya es mínima: **más complejo no siempre vale la pena**.

Límite

Todos siguen sin entender el **tope en 5.0**. Más adelante, árboles y ensambles captarán mejor estas no linealidades sin tener que elegir el grado.

Resumen de la sesión

Ideas clave

- **OLS**: minimizar la suma de residuos al cuadrado; con solución cerrada $(X^\top X)^{-1} X^\top y$.
- **Descenso de gradiente**: bajar el error a pasos; por lotes, estocástico o por minilotes. Requiere **variables escaladas** y buena tasa η.
- **Polinomios**: variables nuevas (x^2, $x_1 x_2 \ldots$); el grado fija la flexibilidad.

Para llevarse

- **Sesgo–varianza**: subajuste (rígido) vs. sobreajuste (sensible). Se elige el grado con **validación**.
- **Métricas**: MAE (promedio), RMSE (castiga errores grandes), R^2 (frente al promedio). Siempre en datos **no vistos**.
- Siempre: **línea base**, `Pipeline`, y revisar residuos.

Siguiente sesión

Regresión regularizada: Ridge, Lasso y Elastic Net para controlar la varianza.

Actividad de laboratorio (individual)

Predicción de precios de viviendas con modelos lineales y polinomiales

Cuaderno de Jupyter con `fetch_california_housing`. Trabaja **de forma individual**.

1. Carga los datos, quita los valores extremos y haz la división entrenamiento/prueba (80/20, `random_state=42`).
2. Calcula la **línea base** (predecir el promedio) con MAE, RMSE y R^2.
3. Ajusta una regresión lineal con `MedInc` y otra con las 7 variables; interpreta los coeficientes.
4. Implementa **descenso de gradiente por lotes** con NumPy y compara con `LinearRegression`; prueba tres tasas η.
5. Compara `SGDRegressor` (con y sin `StandardScaler`).
6. Con un `Pipeline`, ajusta polinomios de grado 1 a 5; grafica el error de entrenamiento y de validación cruzada.
7. Elige el mejor grado, evalúa **una sola vez** en prueba y analiza los residuos.

Entregable: el cuaderno con tus respuestas a preguntas de interpretación (¿hay sobreajuste?, ¿qué métrica reportarías y por qué?).



¿Preguntas?

¡Pasemos a la actividad: predecir precios de vivienda!
